Comandos Útiles para el Parcial

1. Cargar y explorar datos (pandas)

```
import pandas as pd
import numpy as np

df = pd.read_csv("archivo.csv")
df.head()
df.shape
df.columns
df.dtypes
df.info()
df.describe()
df.isna().sum()

df.groupby("col").mean()
df.groupby("col")["otra"].sum()
df.groupby("col").agg(["mean", "std", "count"])
df.sort_values("col", ascending=False)

df.rename(columns =, index =)
df.map()
```

2. NumPy útil

```
np.mean(x)
np.var(x)
np.std(x)
np.sqrt(x)
np.log(x)
```

3. Listas por comprensión

```
[f(c) for c in list]
[f(c) for c in list if condicion]
```

4. Funciones lambda

```
lambda x : f(x)
lambda x, y : f(x,y)
```

```
# Estructura if-else
lambda x: f(x) if condicion else g(x)
```

5. Regresión Lineal (sklearn)

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit()
model.coef_
model.intercept_
model.predict()
model.score()
```

6. Ridge

```
from sklearn.linear_model import Ridge

model = Ridge(alpha=1.0)
model.fit()
```

6. Formulaic

```
from formulaic import Formula
y, X = Formula('y ~ x1 + poly(x2, grado, raw=True)').get_model_matrix(df)
```

7. Train / Test

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)
```

8. Métricas

```
from sklearn.metrics import mean_squared_error, r2_score

mean_squared_error()
r2_score()
```

9. Escalamiento

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

scaler = MinMaxScaler()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = scaler.fit(X)
X_scaled = scaler.transform(X)
```

10. PCA

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
Z_pca = pca.fit_transform(X)
pca.components_
pca.explained_variance_ratio_
```

11. SQL típico

```
SELECT col, COUNT(*)
FROM tabla
WHERE condicion
GROUP BY col
ORDER BY col DESC;



-- INNER JOIN
SELECT t1.col1, t2.col2
FROM tabla1 AS t1
INNER JOIN tabla2 AS t2
ON t1.id = t2.id;



-- LEFT JOIN
SELECT t1.col1, t2.col2
FROM tabla1 AS t1
LEFT JOIN tabla2 AS t2
ON t1.id = t2.id;



# --- CSV → SQLite ---

import pandas as pd
```

```
import sqlite3

con = sqlite3.connect(":memory:")

df = pd.read_csv("archivo.csv")

df.to_sql("tabla", con, if_exists="replace", index=False)

# --- SELECT ---

df2 = pd.read_sql_query("SELECT * FROM tabla", con)

con.close()
```

12. Seaborn y Seaborn Objects

```
import seaborn as sns
sns.boxplot()

import seaborn.objects as so

so.Plot(df, x="x", y="y", color="grupo").add(so.Dot())
so.Plot(df, x="x", y="y", color="grupo").add(so.Line())
so.Plot(df, x="x", color="grupo").add(so.Bars(), so.Hist())
so.Plot(df, x="grupo", y="y", color="grupo").add(so.Bar(), so.Agg("mean"))
so.Plot(df, x="x", y="y", color="grupo").add(so.Line(), so.Agg("mean"))
```

12. Clustering y clasificación

```
# --- KMeans ---
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=K, random_state=0, n_init="auto")
Metodos: fit, predict, fit_predict

# --- DBSCAN ---
from sklearn.cluster import DBSCAN

db = DBSCAN(eps=0.5, min_samples=5)
Metodos: fit, predict, fit_predict

# --- KNN ---
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
```

Metodos: fit, predict